

POLITECNICO

MILANO 1863

PROVA FINALE

PROGETTO DI RETI LOGICHE

Prof. William Fornaciari

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Anno accademico: 2023/2024

Alessandro Del Fatti

Codice persona: 10790553

Consegna del 01/04/2024

Indice

1	Introduzione	2
2	Architettura	3
2.1	Implementazione in VHDL	4
2.2	Macchina a stati finiti	5
2.2.1	INIT	6
2.2.2	READY	6
2.2.3	MEM_READ	7
2.2.4	W_MEM_PREP	7
2.2.5	W_UPDATE	7
2.2.6	C_MEM_PREP	7
2.2.7	C_UPDATE	7
2.2.8	DONE	7
3	Risultati sperimentali	7
3.1	Test basati sugli esempi delle specifiche	8
3.1.1	Esempi 1, 4, 5	8
3.1.2	Esempio 2	8
3.1.3	Esempio 3	8
3.2	Test su casi limite - valori	8
3.2.1	Test 1: valori massimi	8
3.2.2	Test 2: valori minimi	8
3.2.3	Test 3: valore minimo di elaborazioni da eseguire	8
3.2.4	Test 4: valore massimo di elaborazioni da eseguire	9
3.2.5	Test 5: indirizzo di partenza uguale a 0	9
3.3	Test su casi limite - comportamenti	9
3.3.1	Test 1: più segnali di start senza dare il reset	9
3.3.2	Test 2: reset durante l'elaborazione	9
3.3.3	Test 3: start durante il reset	9
4	Conclusioni	9

1 Introduzione

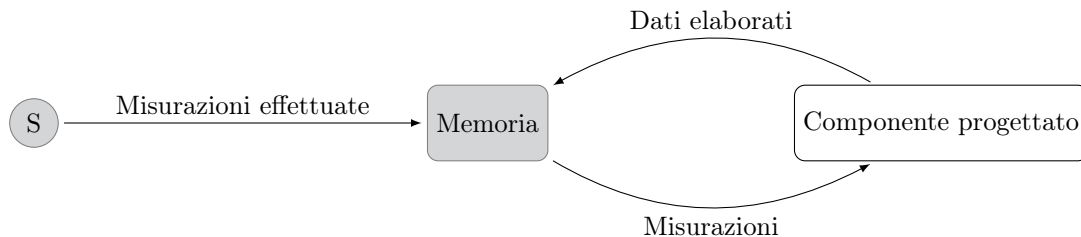
Il componente progettato ha come scopo quello di ricevere in input degli interi positivi, valutarne la loro attendibilità seguendo le regole riportate nei punti sottostanti e generare, in modo coerente con il dato numerico analizzato, un valore intero che possa descriverne la credibilità. Si pensi, ad esempio, ad un sistema che opera in questo modo:

1. Un sensore S effettua una serie di misurazioni: sia K il numero di misurazioni effettuate.
2. Le K misurazioni sono memorizzate in delle celle di memoria. Sia W il contenuto di ciascuna cella di memoria rappresentante una misurazione.
3. Per ciascuna delle K misurazioni si vuole stabilire se il valore memorizzato è attendibile oppure no. Si considerano attendibili valori di $W > 0$ e non attendibili valori di $W = 0$.
4. Per fare ciò, per ciascuna misurazione W_i salvata in una cella di memoria ADD_i , si assegna un valore di credibilità C_i , memorizzandolo nella cella successiva, ADD_{i+1} , seguendo queste regole:
 - (a) Se $W_i > 0$, il corrispondente valore di credibilità C_i sarà massimo, cioè $C_i = 31$
 - (b) Se $W_i = 0$, la misurazione è da considerarsi non valida. Si sostituisce quindi il valore di W_i con W_{i-1} e si pone:

$$C_i = \begin{cases} C_{i-1} - 1, & \text{se } C_{i-1} \geq 1 \\ 0, & \text{se } C_{i-1} = 0 \end{cases}$$

Nel caso in cui si stia elaborando il primo valore della sequenza e perciò W_{i-1} e C_{i-1} non sono definiti, non si può effettuare la «correzione» appena descritta e, di conseguenza, si mantengono $W_0 = 0$ e $C_0 = 0$.

Il seguente schema esplicita in modo generale il principio di funzionamento del sistema e dove si colloca il componente progettato.

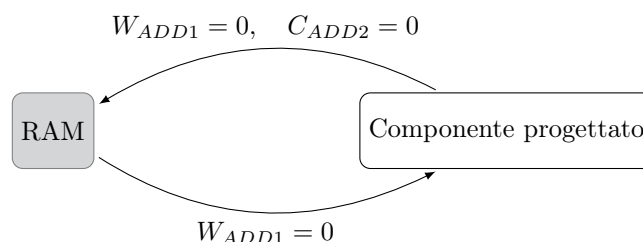


Un esempio di elaborazione, con valori numerici, che racchiude le casistiche di input descritte nell'introduzione è il seguente:

Indirizzo	Valore pre-elaborazione	Valore post-elaborazione
ADD_1	0	0
ADD_2	0	0
ADD_3	192	192
ADD_4	0	31
ADD_5	0	192
ADD_6	0	30

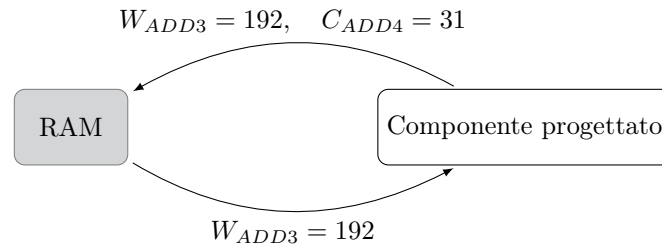
Tabella 1: Situazione della memoria pre e post-elaborazione

Passo 1: Elaborazione del primo valore



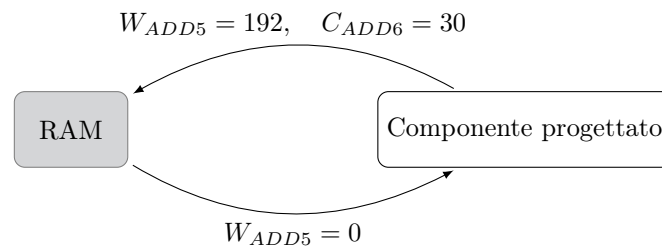
Viene letto $W_{ADD_1} = 0$ dalla memoria. Si tratta di un valore nullo ed è il primo della sequenza, quindi non può essere «corretto» con il valore all'indirizzo precedente. Il componente mantiene quindi $W_{ADD_1} = 0$ e pone $C_{ADD_2} = 0$.

Passo 2: Elaborazione del secondo valore



Viene letto $W_{ADD_3} = 192$ dalla memoria. Si tratta di un valore non nullo. Il componente mantiene quindi $W_{ADD_3} = 192$ e pone $C_{ADD_4} = 31$.

Passo 3: Elaborazione del terzo valore



Viene letto $W_{ADD_5} = 0$ dalla memoria. Si tratta di un valore nullo, ma non è il primo della sequenza, quindi può essere «corretto» con il valore all'indirizzo precedente. Il componente pone quindi $W_{ADD_5} = W_{ADD_3} = 192$ e pone $C_{ADD_6} = C_{ADD_4} - 1 = 30$.

I valori numerici di questo esempio sono coerenti con le specifiche: K , cioè il numero di letture che il componente deve effettuare, è esprimibile con 10 bit. W , il dato in memoria, è una parola da 8 bit e gli indirizzi di memoria sono da 16 bit.

2 Architettura

Il componente progettato, mostrato in Figura 1, è stato suddiviso in sotto-blocchi funzionali, ognuno con un compito diverso:

- **ADDR_REG**, è un registro che mantiene memorizzato l'indirizzo della cella di memoria sulla quale effettuare le operazioni di lettura e/o scrittura.
- **K_VAL_REG**, è un registro che viene inizializzato con il valore K fornito in ingresso. Indica quante elaborazioni devono ancora essere effettuate.
- **W_REG**, è un insieme di due registri che mantengono memorizzati, rispettivamente, la parola W e la credibilità C correnti.
- **FSM**, è la macchina a stati che funziona da unità di controllo. Abilita e disabilita le uscite (*e di conseguenza i rispettivi ingressi degli altri componenti*) in funzione dello stato in cui si trova. La descrizione degli stati e dei segnali di uscita è riportata in sezione 2.2.

Nello schema è presente anche un **multiplexer**, necessario per portare in uscita il valore di W o di C in base alla necessità di scrittura.

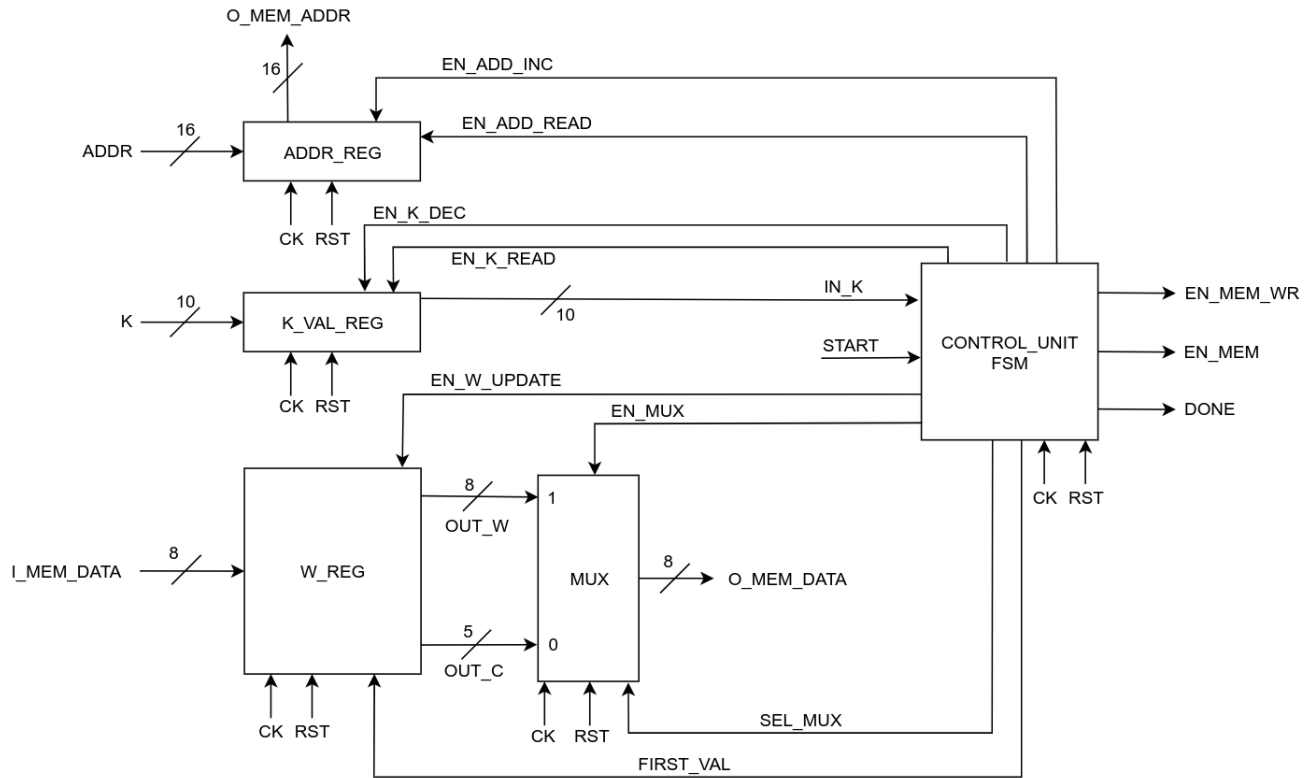


Figura 1: Schema a blocchi del componente progettato

2.1 Implementazione in VHDL

La struttura appena descritta, e riportata in Figura 1 è stata anche mappata nel codice VHDL, di cui si riportano le entity:

```

1 entity addr_reg is
2   port(
3     i_addr      : in std_logic_vector(15 downto 0);
4     i_clk       : in std_logic;
5     i_rst       : in std_logic;
6     en_add_inc  : in std_logic;
7     en_add_read : in std_logic;
8     o_mem_addr  : out std_logic_vector(15 downto 0)
9   );
10 end entity addr_reg;

```

In cui, `i_addr` è un vettore inizializzato con il primo indirizzo di memoria dal quale iniziare l'elaborazione e `o_mem_addr` è l'indirizzo corrente che indica qual è la cella di memoria da leggere o scrivere; viene di volta in volta incrementato (avendo definito un opportuno `signal` nell'architettura relativa a questa entity, dato che `o_mem_addr` è un output e non può essere sovrascritto).

```

1 entity k_val_reg is
2   port(
3     i_k         : in std_logic_vector(9 downto 0);
4     i_clk       : in std_logic;
5     i_rst       : in std_logic;
6     en_k_read   : in std_logic;
7     en_k_dec    : in std_logic;
8     o_k_val     : out std_logic_vector(9 downto 0)
9   );
10 end entity k_val_reg;

```

In cui, i_k è un vettore inizializzato con la lunghezza K della sequenza da elaborare e o_k_val è il valore corrente di K (indica cioè quanti valori devono ancora essere elaborati), che viene decrementato al termine di ogni elaborazione fino a diventare zero. Anche in questo caso, nell'architettura è stato introdotto un apposito **signal** che permette la riscrittura di K .

```

1 entity w_reg is
2   port(
3     i_mem_data    : in std_logic_vector(7 downto 0);
4     i_clk         : in std_logic;
5     i_rst         : in std_logic;
6     i_first_val   : in std_logic;
7     en_w_update   : in std_logic;
8     out_w         : out std_logic_vector(7 downto 0);
9     out_c         : out std_logic_vector(4 downto 0)
10  );
11 end entity w_reg;

```

In cui, i_mem_data è il vettore di 8 bit contenente il dato proveniente dalla memoria e che deve essere elaborato secondo i requisiti di funzionamento specificati nell'introduzione. out_w e out_c sono i due vettori contenenti i valori post-elaborazione, rispettivamente da 8 e 5 bit, e che devono essere scritti in memoria. Anche in questo caso, nell'architettura sono stati definiti due appositi **signal** per permettere la riscrittura di tali output.

```

1 entity mux is
2   port(
3     in_w          : in std_logic_vector(7 downto 0);
4     in_c          : in std_logic_vector(4 downto 0);
5     en_mux        : in std_logic;
6     sel_mux       : in std_logic;
7     i_clk         : in std_logic;
8     i_rst         : in std_logic;
9     o_mem_data    : out std_logic_vector(7 downto 0)
10  );
11 end entity mux;

```

In cui, in_w e in_c sono i segnali provenienti dal blocco W_REG e o_mem_data è il vettore da 8 bit che deve essere scritto in memoria assumendo, alternativamente, il valore di W o di C . Nel secondo caso, il multiplexer si occupa anche di fare un'operazione di padding per portare in_c , vettore da 5 bit, su o_mem_data , vettore da 8 bit.

Tutte le architetture di questi quattro componenti sono formate da un unico **process** che ha i soli segnali di clock e reset nella sensitivity list. L'architettura della macchina a stati, descritta nel seguito, è invece formata da due **process** differenti:

- Un **process** che descrive la **funzione di stato prossimo** e ha nella sensitivity list i segnali di clock e reset;
- Un **process** che descrive la **funzione di uscita** e ha nella sensitivity list il segnale di stato corrente.

2.2 Macchina a stati finiti

Omettendo il codice VHDL della entity dell'FSM, in Figura 2 è invece schematizzato il comportamento di transizione tra i diversi stati della macchina, da cui si possono individuare alcune macro-aree che racchiudono parte degli stati:

- **Preparazione** del componente, contenente gli stati di INIT e di READY. In questi stati l'elaborazione vera e propria non è ancora iniziata, ma vengono gestiti gli opportuni segnali per poter proseguire nell'elaborazione.
- **Lettura**, contenente lo stato di MEM_READ.
- **Aggiornamento**, contenente i quattro stati che si occupano di elaborare ciò che è stato letto dalla memoria e scriverne i corretti valori.

- **Termine** dell'elaborazione, contenente lo stato di DONE.

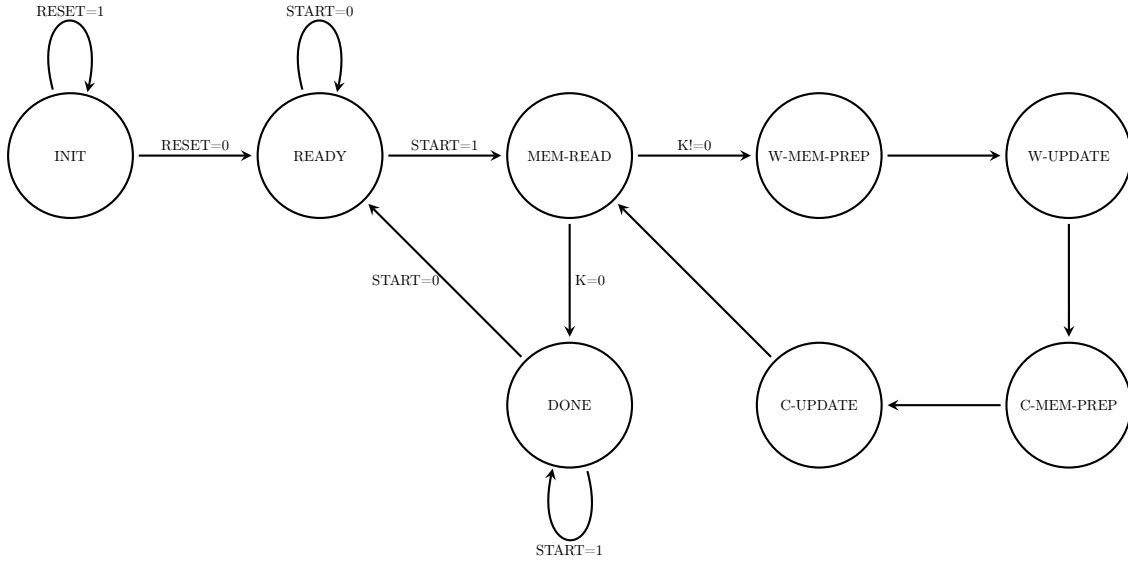


Figura 2: Macchina a stati finiti

Essendo una macchina di Moore, a ciascuno stato sono anche specificate delle uscite. Per una migliore leggibilità, il valore delle uscite è riportato in Tabella 2.

	o_done	o_en_mux	o_sel_mux	o_w_update	o_k_read	o_k_dec	o_add_read	o_add_inc	o_en_mem	o_mem_wr	o_first_val
INIT	0	0	0	0	0	0	0	0	0	0	0
READY	0	0	0	0	1	0	1	0	0	0	1
MEM_READ	0	0	0	0	0	0	0	0	1	0	0
W_MEM_PREP	0	0	0	1	0	0	0	0	0	0	0
W_UPDATE	0	1	1	0	0	0	0	0	0	0	0
C_MEM_PREP	0	1	0	0	0	0	0	1	1	1	0
C_UPDATE	0	0	0	0	0	1	0	1	1	1	0
DONE	1	0	0	0	0	0	0	0	0	0	0

Tabella 2: Uscite per ciascuno stato della FSM

Di seguito sono invece descritti i vari stati:

2.2.1 INIT

Lo stato di INIT è quello che il componente raggiunge non appena viene dato il segnale di **reset**. Sarà quindi lo stato iniziale della prima elaborazione.

2.2.2 READY

Lo stato di READY si raggiunge non appena il segnale di **reset** viene portato a zero. Rappresenta lo stato in cui la macchina è in attesa del segnale alto di **start**, per poter iniziare l'elaborazione. Mentre il componente è in questo stato, inoltre, inizia la lettura di *K* e dell'indirizzo iniziale di memoria (sono infatti abilitate le uscite *o_k_read* e *o_add_read*). Infine, in questo stato, viene abilitata l'uscita *o_first_val* per l'inizializzazione del registro *W_REG* che pone a 0 i valori di *W* e di *C*.

2.2.3 MEM_READ

Lo stato di MEM_READ valuta il valore di K per capire se bisogna continuare con l'elaborazione oppure terminare nello stato di DONE. Contestualmente, abilita l'uscita `o_en_mem` per la lettura della memoria (preparandosi quindi a ricevere il nuovo valore da elaborare).

2.2.4 W_MEM_PREP

Lo stato di W_MEM_PREP si raggiunge se $K > 0$. In questo stato viene abilitata l'uscita `o_w_update` per consentire al componente di valutare, ed eventualmente aggiornare, il valore di W .

2.2.5 W_UPDATE

Lo stato di W_UPDATE fa terminare il processo di aggiornamento del valore di W che dovrà essere scritto in memoria abilitando le uscite della macchina a stati `o_en_mux` e `o_sel_mux` per permettere di portare su `o_mem_data`, al colpo di clock successivo, il nuovo valore di W da scrivere in memoria.

2.2.6 C_MEM_PREP

Lo stato di C_MEM_PREP abilita le uscite `o_en_mem` e `o_mem_wr` per poter scrivere in memoria il valore appena portato su `o_mem_data`. Contestualmente, attiva anche l'uscita `o_add_inc` per poter incrementare l'indirizzo corrente e l'uscita `o_en_mux` per portare su `o_mem_data` il nuovo valore di credibilità memorizzato in W_REG.

2.2.7 C_UPDATE

Lo stato di C_UPDATE abilita le uscite `o_en_mem` e `o_mem_wr` per poter scrivere in memoria il valore della credibilità appena portato su `o_mem_data`. Contestualmente, abilita l'uscita `o_add_inc` per l'incremento dell'indirizzo in vista della successiva lettura e `o_k_dec` per il decremento di K , dato che l'elaborazione di questa coppia di valori si è conclusa.

2.2.8 DONE

Lo stato di DONE si raggiunge quando $K = 0$, cioè quando l'elaborazione è terminata. Perciò viene abilitata l'uscita `o_done`. Si resta in questo stato fino a che `start` rimane alto. Non appena `start` viene portato a 0, la macchina torna di nuovo in uno stato di READY.

3 Risultati sperimentali

Per testare il componente progettato, come primo passo, è stato sottoposto al testbench fornito insieme alla specifica. Completata correttamente la sintesi e terminate le due simulazioni, behavioral e post-synthesis sono stati generati i `report_timing` e `report_utilization` con i seguenti risultati:

- **Slack (MET):** 16.162ns (`required time - arrival time`). Sono quindi stati rispettati i vincoli temporali imposti dalla specifica.
- **Register as Flip Flop:** 50, **Register as Latch:** 0. Da cui si nota che sono stati utilizzati 50 flip-flop e nessun latch. Dallo stesso report si può anche vedere il numero di look-up table utilizzate, in questo caso 52.

Una volta terminata senza errori la simulazione del testbench fornito, il secondo passo è stato quello di ricostruire i testbench sulla base degli esempi dello stato della memoria presenti nel file delle specifiche. Come terzo passo nella fase di test sono stati individuati dei casi limite legati sia ai valori assumibili dai vari segnali, sia al comportamento del componente. Infine, come ultimo controllo, sono state generate, tramite un apposito script, 100 sequenze di valori pseudo-casuali in cui la probabilità di generare il valore 0 è maggiore degli altri interi così da verificare più approfonditamente il comportamento del componente.

Di seguito, per ciascun test effettuato, sono descritte le peculiarità che hanno portato alla scelta di sottoporre il componente a tale test.

3.1 Test basati sugli esempi delle specifiche

3.1.1 Esempi 1, 4, 5

- $K = 10$ e $K = 16$ elaborazioni da eseguire
- Primo valore non nullo
- Presenza di valori nulli interni alla sequenza

3.1.2 Esempio 2

- $K = 35$ elaborazioni da eseguire
- Primo valore non nullo
- Presenza di valori nulli interni alla sequenza. In particolare sono presenti più di 31 valori nulli. Il test va quindi a verificare che, raggiunto un valore di credibilità pari a 0, il componente continua a comportarsi correttamente lasciando nullo il valore di credibilità.

3.1.3 Esempio 3

- $K = 33$ elaborazioni da eseguire
- Primo valore nullo. Il test va quindi a verificare che, letto il primo 0, il valore che viene scritto in memoria sia 0 sia per la parola W , sia per la credibilità C .
- Presenza di valori nulli interni alla sequenza

3.2 Test su casi limite - valori

3.2.1 Test 1: valori massimi

- $K = 10$ elaborazioni da eseguire
- Primo valore non nullo
- Non ci sono valori nulli interni alla sequenza

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora i valori di W siano tutti massimi, ovvero in memoria ci siano solo valori pari a 255.

3.2.2 Test 2: valori minimi

- $K = 32$ elaborazioni da eseguire
- Primo valore nullo
- Presenza di valori nulli interni alla sequenza

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora i valori di W siano tutti minimi, ovvero in memoria ci siano solo valori pari a 0.

3.2.3 Test 3: valore minimo di elaborazioni da eseguire

- $K = 1$ elaborazioni da eseguire
- Valore non nullo

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora il numero di elaborazioni da eseguire sia minimo, ovvero $K = 1$.

3.2.4 Test 4: valore massimo di elaborazioni da eseguire

- $K = 1023$ elaborazioni da eseguire
- Valori della sequenza generati in modo pseudo-casuale

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora il numero di elaborazioni da eseguire sia massimo, ovvero $K = 1023$.

3.2.5 Test 5: indirizzo di partenza uguale a 0

- $K = 10$
- Indirizzo di partenza $ADD = 0$
- Valori della sequenza generati in modo pseudo-casuale

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora l'indirizzo di partenza da cui partire con l'elaborazione sia 0.

3.3 Test su casi limite - comportamenti

3.3.1 Test 1: più segnali di start senza dare il reset

- Più segnali di start forniti
- Indirizzi di partenza uguali per due sequenze
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di permettere al componente di effettuare più elaborazioni senza passare dallo stato di reset, se non per la prima sequenza. Il componente deve quindi completare le elaborazioni di tutte le sequenze. Inoltre, due sequenze partono dallo stesso indirizzo di memoria e, perciò, il componente deve comportarsi correttamente sovrascrivendo valori già elaborati.

3.3.2 Test 2: reset durante l'elaborazione

- Più segnali di start forniti
- Segnale di reset fornito durante una elaborazione
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di permettere al componente di effettuare più elaborazioni. Durante una elaborazione viene fornito il segnale di reset e il componente deve interrompere l'elaborazione tornando nello stato di INIT fino a che il reset viene portato a 0.

3.3.3 Test 3: start durante il reset

- Segnale di start e di reset contemporaneamente alti
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di verificare che, mentre il segnale di reset è alto, nel caso in cui venga fornito il segnale di start, il componente non inizi l'elaborazione ma rimanga in uno stato di reset.

4 Conclusioni

Il componente progettato soddisfa i requisiti temporali imposti dalle specifiche e simula correttamente i casi di test sopra elencati, sia in behavioral che in post-sintesi.

Come mostrato dallo schema a blocchi riportato in Figura 1 ho scelto di progettare il componente suddividendolo in vari blocchi funzionali. Questa scelta ha semplificato non solo la fase di progettazione iniziale, e la conseguente scrittura del codice VHDL, ma anche la gestione dei vari segnali per garantire il funzionamento nei diversi stati di elaborazione.